Taller 4: Grafos – Misión Lázaro

Saul Leonardo Cruz Romero[1*], Isabella Hermosa Losada[1] and Alejandro Macias Barrios[1]

[1*] Departamento de Ingeniería de Sistemas, Pontificia Universidad Javeriana, Bogotá, Colombia .

*Corresponding author(s). E-mail(s): sa.cruz@javeriana.edu.co;
Contributing authors: ishermosa@javeriana.edu.co;
amaciasb@javeriana.edu.co;

Abstract

En este taller de Estructuras de Datos se desarrolla una solución para el problema de exploración interplanetaria de la Misión Lázaro mediante el uso de grafos. Cada planeta se representa como un vértice y las conexiones entre planetas se modelan como aristas ponderadas según el combustible necesario para realizar cada desplazamiento.

La implementación utiliza matrices para almacenar los pesos y las conexiones gravitacionales peligrosas, además de una colección con la información de los planetas. Para determinar el recorrido de la nave se emplea una adaptación del algoritmo de Prim que prioriza las conexiones seguras y considera el consumo de combustible.

Finalmente, se generan archivos con el detalle del recorrido y un resumen de la misión, y se comprueba el funcionamiento mediante diferentes casos de prueba.

Keywords: grafos, matriz de adyacencia, algoritmo de Prim, estructuras de datos, C++

1 Introducción

Los grafos permiten representar problemas en los que existen elementos relacionados mediante conexiones. En el contexto de una exploración interplanetaria, esta estructura resulta útil para modelar los planetas disponibles, los posibles desplazamientos entre ellos y los costos asociados a cada recorrido.

En el Taller 4 de Estructuras de Datos se desarrolla la Misión Lázaro, cuyo objetivo es determinar un recorrido que permita a una nave explorar los planetas registrados y regresar a la Tierra.

Para realizar esta exploración se deben considerar las distancias entre los planetas, sus factores temporales, el combustible necesario para desplazarse y la existencia de conexiones gravitacionales peligrosas.

El problema se representa mediante un grafo no dirigido. Cada vértice representa un planeta y cada arista representa una conexión posible entre dos planetas. El peso de las aristas corresponde al combustible requerido para realizar el desplazamiento.

Además, se identifica cuáles conexiones son peligrosas, ya que esta información influye en las decisiones de la nave.

Para almacenar la información se utiliza una colección de planetas junto con dos matrices: una para los pesos de las conexiones y otra para identificar su peligrosidad.

La solución se organiza mediante tres tipos abstractos de datos (TAD): Planeta, Grafo y Sistema de Exploración Lázaro, separando la representación de los datos de las operaciones necesarias para desarrollar la misión.

El recorrido se determina mediante una adaptación del algoritmo de Prim. A diferencia de su aplicación tradicional para construir un árbol de expansión mínima, en este taller se selecciona el siguiente destino desde la posición actual de la nave, dando prioridad a las conexiones seguras y escogiendo la de menor consumo de combustible entre las opciones del mismo tipo.

Una vez explorados los planetas, la nave regresa directamente a la Tierra.

Finalmente, el programa genera dos archivos de texto: uno con el detalle de los desplazamientos y otro con el resumen de la misión.

Su funcionamiento se comprueba mediante casos de prueba que permiten verificar el recorrido, los consumos de combustible y el tratamiento de las conexiones peligrosas.

Desarrollo

4.1 Diseño del grafo

Para representar la exploración interplanetaria utilizamos un grafo no dirigido y ponderado. Cada vértice representa uno de los planetas registrados, incluida la Tierra, que es el punto de partida y de regreso de la nave.

Las aristas representan las conexiones por las que la nave puede desplazarse entre dos planetas.

En este problema, todos los planetas pueden conectarse entre sí, pero algunas conexiones están identificadas como gravitacionalmente peligrosas.

Como los desplazamientos son bidireccionales, el peso y la peligrosidad de una conexión son iguales en ambos sentidos.

4.1.1 Cálculo de los pesos

El peso de cada arista corresponde a la cantidad de combustible necesaria para viajar entre los dos planetas.

Este valor se calcula a partir de la distancia entre sus coordenadas, los factores temporales y las constantes proporcionadas en el archivo de entrada.

Primero calculamos la distancia euclidiana entre los planetas i y j:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} \tag{1}$$

Después calculamos el factor gravitacional que modifica el consumo de combustible:

$$G_{ij} = 1 + \lambda \left(\frac{(t_i - 1) + (t_j - 1)}{2} \right) \tag{2}$$

Finalmente, el combustible necesario para realizar el desplazamiento se obtiene mediante:

$$F_{ij} = k \cdot d_{ij} \cdot G_{ij} \tag{3}$$

En estas expresiones, d_{ij} es la distancia entre los planetas, t_i y t_j son sus factores temporales, λ determina la influencia de esos factores y k es la constante de consumo de combustible.

El valor F_{ij} es el peso que almacenamos en el grafo.

4.1.2 Representación del grafo

Para almacenar la información utilizamos un vector de planetas y dos matrices de adyacencia: una para el combustible de las conexiones y otra para identificar cuáles son peligrosas.

Cada posición del vector identifica un planeta, y ese mismo índice se utiliza para consultar sus conexiones en las matrices.

En nuestra implementación se manejan tres estructuras principales:

- Un vector de planetas que guarda los nombres, las coordenadas y los factores temporales.
- Una matriz de números que almacena el combustible necesario para viajar entre cada par de planetas.
- Una matriz de booleanos que indica si una conexión es peligrosa o segura.

La matriz de combustible es simétrica porque el costo entre dos planetas es el mismo en ambos sentidos.

La diagonal contiene ceros, ya que no se considera un desplazamiento de un planeta hacia sí mismo.

La matriz de peligrosidad también es simétrica. El valor verdadero indica una conexión peligrosa y el valor falso indica una conexión segura.

El archivo de entrada solamente enumera las conexiones peligrosas; las demás se consideran seguras.

Elegimos esta representación porque permite consultar directamente el combustible y la peligrosidad de una conexión utilizando los índices de los planetas.

Además, facilita comparar los posibles destinos desde la posición actual de la nave.

Como desventaja, las matrices necesitan almacenar una posición para cada par de planetas, aunque la información de una conexión sea igual en ambos sentidos.

Su espacio crece de forma cuadrática respecto a la cantidad de planetas.

4.1.3 Ejemplo de almacenamiento

Para mostrar cómo se guarda la información utilizamos los cuatro planetas del archivo `prueba_03_peligros.txt`.

En este caso, $k = 1$ y $\lambda = 0.2$, y todos los planetas tienen factor temporal igual a 1.

La Tabla 1 muestra los datos de los planetas y los índices que utilizamos para relacionarlos con las matrices.

Table 1 Planetas utilizados en el ejemplo.

Índice	Planeta	x	y	z	t
0	Tierra	0	0	0	1
1	Miller	1	0	0	1
2	Mann	3	0	0	1
3	Edmunds	6	0	0	1

Como todos los factores temporales son iguales a 1, el factor gravitacional resulta igual a 1 para cada conexión.

Además, como $k = 1$, el combustible coincide numéricamente con la distancia entre los planetas.

Por ejemplo, entre Tierra y Miller la distancia es 1, por lo que el combustible necesario también es 1.

De la misma manera se calculan los pesos de las demás conexiones.

La Figura 1 muestra las seis conexiones entre los cuatro planetas. Cada número indica el combustible necesario para viajar entre ese par de planetas; las líneas continuas son conexiones seguras y las discontinuas identifican las peligrosas.

A partir de las conexiones de este grafo se obtienen dos matrices: una con los pesos de combustible y otra con la peligrosidad.

La matriz de combustible obtenida es:

$$P = \begin{pmatrix} 0 & 1 & 3 & 6 \\ 1 & 0 & 2 & 5 \\ 3 & 2 & 0 & 3 \\ 6 & 5 & 3 & 0 \end{pmatrix} \tag{4}$$

Las filas y columnas siguen el orden de los planetas mostrado en la Tabla 1.

En este archivo, las conexiones Tierra–Miller y Miller–Mann son peligrosas.

La matriz de peligrosidad, representada con 1 para verdadero y 0 para falso, queda de la siguiente manera:

$$Q = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \tag{5}$$

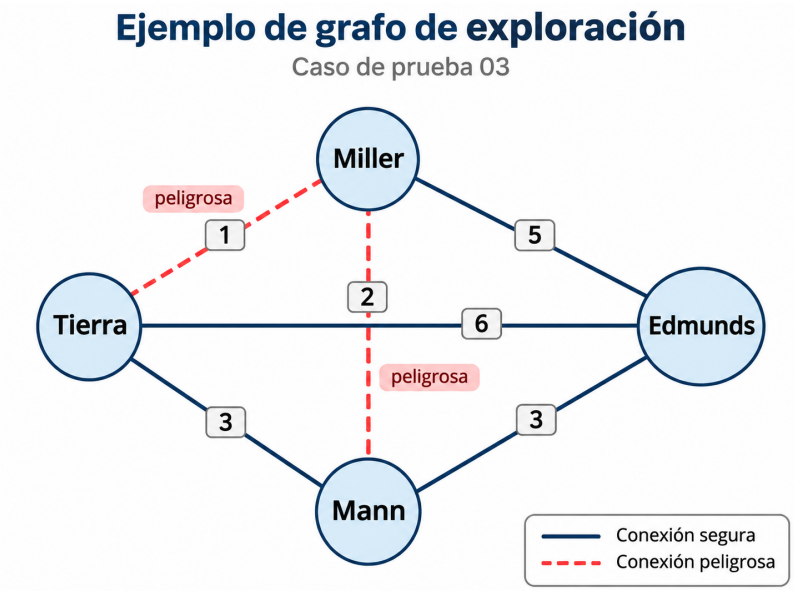


Fig. 1 Grafo ponderado del caso de prueba 03, con conexiones seguras y peligrosas.
Nota: imagen generada con apoyo de inteligencia artificial a partir de un dibujo elaborado por la autora.

Por ejemplo, la posición $(0, 1)$ de la matriz P contiene el valor 1, que corresponde al combustible necesario para viajar de Tierra a Miller.

En esa misma posición, la matriz Q contiene un 1, indicando que la conexión es peligrosa.

En cambio, la posición $(0, 2)$ de la matriz Q contiene un 0, porque la conexión entre Tierra y Mann es segura, aunque su consumo de combustible sea mayor.

Este ejemplo muestra cómo se relacionan los datos de los planetas con las matrices de adyacencia y cómo se almacenan tanto los pesos como la peligrosidad de las conexiones.

4.1.4 Diseño de los TAD

Para organizar la información del grafo se definieron tres TAD: Planeta, Grafo y Sistema de Exploración Lázaro.

A continuación se describe qué información almacena cada uno, cuáles son sus condiciones y qué operaciones permite realizar.

4.1.5 TAD Planeta

Representa uno de los planetas de la misión y conserva su nombre, sus coordenadas espaciales y su factor temporal.

Esta información permite identificar cada planeta y calcular el combustible necesario para desplazarse hacia los demás.

Estado

Nombre, cadena de caracteres. Identifica al planeta dentro de la misión.

- **Condiciones de la variable:** Su valor inicial es una cadena vacía. Para registrar el planeta en el grafo, debe tener un nombre que no esté repetido.

Coordenada X, número decimal. Representa la posición del planeta sobre el primer eje del espacio.

- **Condiciones de la variable:** Su valor inicial es cero. Puede tomar valores positivos, negativos o iguales a cero.

Coordenada Y, número decimal. Representa la posición del planeta sobre el segundo eje del espacio.

- **Condiciones de la variable:** Su valor inicial es cero. Puede tomar valores positivos, negativos o iguales a cero.

Coordenada Z, número decimal. Representa la posición del planeta sobre el tercer eje del espacio.

- **Condiciones de la variable:** Su valor inicial es cero. Puede tomar valores positivos, negativos o iguales a cero.

Factor temporal, número decimal. Representa el valor temporal asociado al planeta, que se utiliza para ajustar el combustible necesario en los desplazamientos.

- **Condiciones de la variable:** Su valor inicial es uno. Para registrar el planeta en el grafo, el factor temporal debe ser mayor o igual a uno.

Interfaz

Crear planeta: Planeta. Inicia un planeta con sus valores iniciales.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Se obtiene un planeta con el nombre vacío, las tres coordenadas en cero y el factor temporal igual a uno.

Consultar nombre: cadena de caracteres. Permite conocer el nombre del planeta.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Se obtiene el nombre almacenado. Los demás datos del planeta conservan sus valores.

Consultar coordenada X: número decimal. Permite conocer la posición del planeta en el primer eje espacial.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Se obtiene el valor de la coordenada X. Los datos del planeta no se modifican.

Consultar coordenada Y: número decimal. Permite conocer la posición del planeta en el segundo eje espacial.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Se obtiene el valor de la coordenada Y. Los datos del planeta no se modifican.

Consultar coordenada Z: número decimal. Permite conocer la posición del planeta en el tercer eje espacial.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Se obtiene el valor de la coordenada Z. Los datos del planeta no se modifican.

Consultar factor temporal: número decimal. Permite conocer el factor temporal asociado al planeta.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Se obtiene el factor temporal almacenado. Los datos del planeta conservan sus valores.

Asignar nombre: vacío. Modifica el nombre del planeta utilizando el nuevo nombre recibido.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** El nombre queda reemplazado por el valor recibido. Las coordenadas y el factor temporal conservan sus valores.

Asignar coordenadas: vacío. Modifica las tres coordenadas espaciales del planeta con los valores recibidos.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Las coordenadas X, Y y Z quedan reemplazadas por los nuevos valores. El nombre y el factor temporal no se modifican.

Asignar factor temporal: vacío. Modifica el factor temporal asociado al planeta.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** El factor temporal queda reemplazado por el valor recibido. El nombre y las coordenadas conservan sus valores.

4.1.6 TAD Grafo

Representa los planetas de la misión y sus conexiones.

Almacena el combustible necesario para viajar entre ellos e identifica las conexiones peligrosas.

Estado

Planetas, secuencia de planetas. Almacena los planetas registrados en la misión.

- **Condiciones de la variable:** Inicialmente está vacía. No contiene planetas con nombres vacíos o repetidos ni con factores temporales menores que uno. Conserva el orden de registro.

Pesos, matriz de números decimales. Almacena el combustible de las conexiones entre los planetas.

- **Condiciones de la variable:** Inicialmente está vacía. Tiene una fila y una columna por planeta. Sus valores iniciales son cero y los pesos son iguales en ambos sentidos.

Conexiones peligrosas, matriz de booleanos. Indica cuáles conexiones entre planetas son gravitacionalmente peligrosas.

- **Condiciones de la variable:** Inicialmente está vacía. Tiene las mismas dimensiones que la matriz de pesos. Las conexiones comienzan en falso y una conexión peligrosa se marca en ambos sentidos.

Interfaz

Crear grafo: Grafo. Inicia un grafo sin planetas ni conexiones registradas.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Se obtiene un grafo con la secuencia de planetas y las dos matrices vacías.

Agregar planeta: booleano. Intenta registrar un planeta en el grafo.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Si el planeta tiene un nombre no vacío y no repetido, y su factor temporal es mayor o igual a uno, se agrega al final de la secuencia y las matrices se amplían para incluirlo. Retorna verdadero.
  En caso contrario, retorna falso y el grafo conserva sus datos.

Consultar cantidad de planetas: número entero. Indica cuántos planetas contiene el grafo.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Retorna la cantidad de planetas registrados. El grafo no se modifica.

Buscar planeta: posición de planeta. Busca un planeta por su nombre.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Retorna la posición del planeta cuando lo encuentra o menos uno cuando no existe. El grafo no se modifica.

Obtener planeta: Planeta. Permite consultar el planeta ubicado en una posición determinada.

- **Precondiciones:** La posición corresponde a un planeta registrado.
- **Poscondiciones:** Se obtiene acceso de solo lectura al planeta indicado. El grafo no se modifica.

Establecer peso: vacío. Guarda el combustible de la conexión entre dos planetas.

- **Precondiciones:** Las posiciones de origen y destino son válidas.
- **Poscondiciones:** El peso recibido queda almacenado en ambos sentidos. Las demás conexiones conservan sus valores.

Obtener peso: número decimal. Consulta el combustible almacenado entre dos planetas.

- **Precondiciones:** Las posiciones de origen y destino son válidas.
- **Poscondiciones:** Retorna el peso de la conexión indicada. El grafo no se modifica.

Marcar conexión peligrosa: vacío. Registra una conexión como peligrosa.

- **Precondiciones:** Las posiciones de origen y destino son válidas.
- **Poscondiciones:** La conexión queda marcada como peligrosa en ambos sentidos. Los pesos no se modifican.

Consultar peligrosidad: booleano. Determina si una conexión está marcada como peligrosa.

- **Precondiciones:** Las posiciones de origen y destino son válidas.
- **Poscondiciones:** Retorna verdadero si la conexión es peligrosa y falso en caso contrario. El grafo no se modifica.

4.1.7 TAD Sistema de Exploración Lázaro

Organiza la misión utilizando la información de los planetas y sus conexiones.

Se encarga de cargar los datos, calcular el combustible, realizar la exploración y generar los archivos con los resultados.

Estado

Grafo, grafo de planetas. Almacena los planetas y sus conexiones, junto con el combustible y la peligrosidad de cada una.

- **Condiciones de la variable:** Inicialmente no contiene planetas. Se construye con la información del archivo de entrada.

Constante de combustible, número decimal. Representa el valor utilizado para calcular el combustible necesario en cada desplazamiento.

- **Condiciones de la variable:** Su valor inicial es cero. El valor leído del archivo de entrada debe ser mayor que cero.

Constante de dilatación temporal, número decimal. Determina cómo influyen los factores temporales en el combustible de las conexiones.

- **Condiciones de la variable:** Su valor inicial es cero. El valor leído del archivo de entrada no puede ser negativo.

Recorrido, secuencia ordenada de planetas. Almacena los planetas en el orden en que la nave los visita, incluido el regreso a la Tierra.

- **Condiciones de la variable:** Inicialmente está vacío. Cuando la exploración termina correctamente, comienza y termina en la Tierra. Los demás planetas aparecen una vez.

Combustible total, número decimal. Acumula el combustible utilizado durante la misión.

- **Condiciones de la variable:** Su valor inicial es cero. Se actualiza con el combustible de cada desplazamiento, incluido el regreso a la Tierra.

Interfaz

Crear sistema de exploración: Sistema de Exploración Lázaro. Inicia el sistema que desarrollará la misión.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Se obtiene un sistema con el grafo y el recorrido vacíos.
  Las dos constantes y el combustible total comienzan en cero.

Cargar planetas: booleano. Lee el archivo de entrada y registra los planetas y las conexiones peligrosas de la misión.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Retorna verdadero cuando se pueden leer los datos y estos cumplen las condiciones necesarias.
  En ese caso, quedan registrados los planetas, las constantes y las conexiones peligrosas.
  Retorna falso si el archivo no puede abrirse o se encuentran datos inválidos.

Calcular combustible: vacío. Calcula el combustible necesario para viajar entre cada par de planetas.

- **Precondiciones:** Los datos de los planetas y las constantes fueron cargados correctamente.
- **Poscondiciones:** Los pesos de las conexiones quedan calculados y registrados en el grafo en ambos sentidos.

Realizar exploración: booleano. Determina el recorrido de la nave desde la Tierra, visitando los demás planetas y regresando al punto de partida.

- **Precondiciones:** Los planetas y las conexiones se encuentran registrados y sus pesos fueron calculados.
- **Poscondiciones:** Si existe la Tierra y se pueden seleccionar los destinos necesarios, retorna verdadero.
  El recorrido queda almacenado y se registra el combustible total consumido.
  Retorna falso cuando no se encuentra la Tierra o no es posible seleccionar otro planeta durante la exploración.

Generar detalle: booleano. Crea un archivo con la información de cada desplazamiento realizado durante la misión.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Retorna verdadero cuando existe un recorrido y el archivo de detalle se genera correctamente.
  El archivo contiene los desplazamientos, las distancias, los factores gravitacionales, el combustible y la peligrosidad de las conexiones utilizadas.
  Retorna falso cuando no hay recorrido o se presenta un error al generar el archivo.

Generar resumen: booleano. Crea un archivo con los resultados generales de la exploración.

- **Precondiciones:** Ninguna.
- **Poscondiciones:** Retorna verdadero cuando existe un recorrido y el archivo de resumen se genera correctamente.
  El archivo presenta el recorrido, la cantidad de desplazamientos, la distancia y el combustible totales, los tipos de conexiones utilizadas y los desplazamientos de mayor y menor consumo.
  Retorna falso cuando no hay recorrido o se presenta un error al generar el archivo.

4.1.8 Relación entre los TAD

La Figura 2 muestra la relación entre los tres TAD utilizados en la misión.

El Sistema de Exploración Lázaro contiene el grafo y utiliza sus operaciones para consultar las conexiones.

A su vez, el grafo almacena los planetas y permite consultar la información de cada uno.

4.2 Implementación del grafo en C++

Para llevar el diseño del grafo a C++ utilizamos las clases Planeta, Grafo y Sistema de Exploración Lázaro. Esta organización permite separar la información de los planetas, sus conexiones y las operaciones necesarias para desarrollar la misión.

La representación se implementó mediante un vector de planetas y dos matrices de adyacencia. La primera matriz almacena el combustible requerido entre cada par de planetas y la segunda indica si una conexión es gravitacionalmente peligrosa.

A continuación se presentan fragmentos del código implementado y se explica cómo se utilizan para registrar los planetas y construir el grafo.

4.2.1 Representación de los planetas

Cada planeta se representa mediante la clase `Planeta`, que conserva su nombre, las coordenadas espaciales y el factor temporal.

Como se observa en la Figura 3, los atributos se declararon privados y se definieron operaciones para consultar y modificar sus valores.

Esta organización permite mantener reunidos los datos que pertenecen a un mismo planeta. Además, otras partes del programa pueden consultar sus coordenadas y su factor temporal sin acceder directamente a los atributos.

Esto es importante al calcular el combustible entre dos planetas, ya que se necesitan sus coordenadas para obtener la distancia y sus factores temporales para determinar el ajuste gravitacional.

4.2.2 Estructuras utilizadas para el grafo

Para representar el grafo implementamos la clase `Grafo`, que almacena los planetas y sus conexiones.

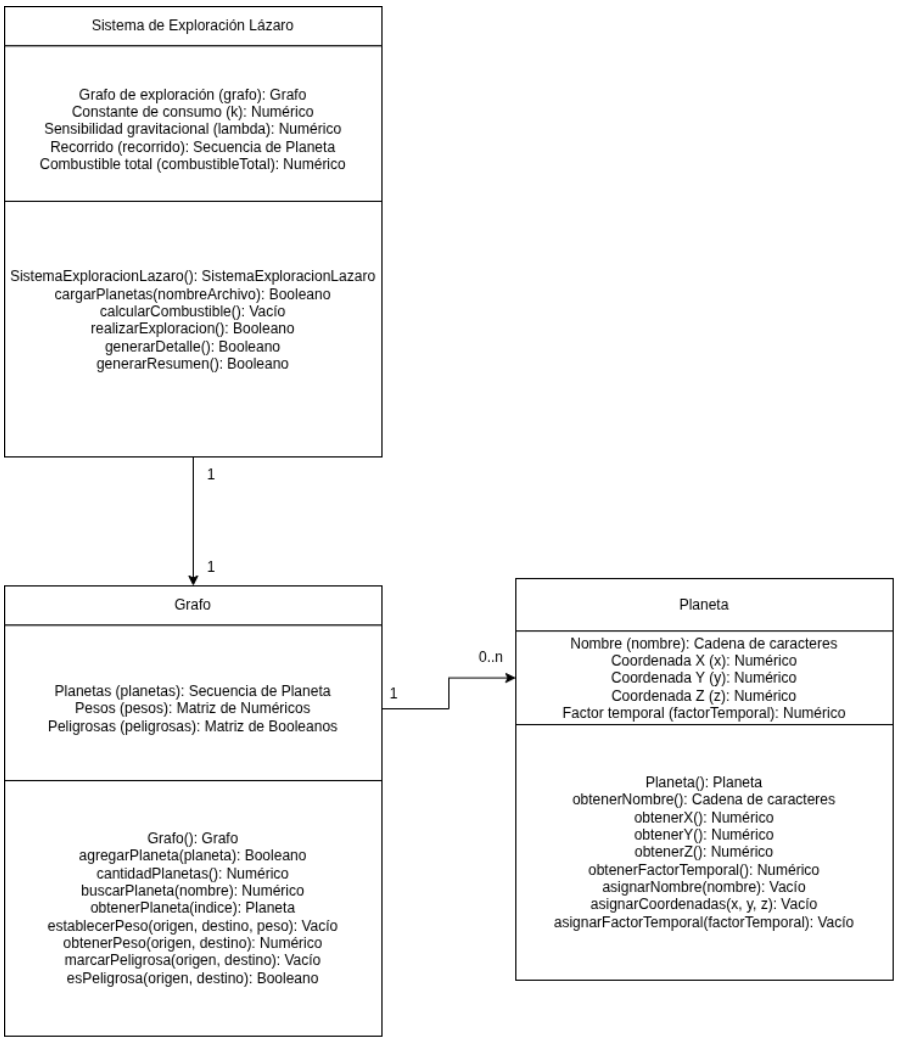


Fig. 2 Relación entre los TAD de la Misión Lázaro.

En la Figura 4 se observan las tres estructuras principales: un vector de objetos Planeta, una matriz para los pesos y otra matriz para identificar las conexiones peligrosas.

El vector conserva los planetas en el orden en que fueron registrados. Cada planeta ocupa una posición que también se utiliza como índice para consultar sus conexiones en ambas matrices.

La matriz de pesos almacena números decimales que representan el combustible necesario para cada desplazamiento. La matriz de peligrosidad almacena valores booleanos que permiten saber si una conexión es segura o peligrosa.

```cpp
#ifndef PLANETA_H
#define PLANETA_H

#include <string>

class Planeta
{
private:
    std::string nombre;
    double x;
    double y;
    double z;
    double factorTemporal;

public:
    Planeta();

    std::string obtenerNombre() const;
    double obtenerX() const;
    double obtenerY() const;
    double obtenerZ() const;
    double obtenerFactorTemporal() const;

    void asignarNombre(const std::string& nuevoNombre);
    void asignarCoordenadas(double nuevaX, double nuevaY, double nuevaZ);
    void asignarFactorTemporal(double nuevoFactorTemporal);
};

#endif
```

Fig. 3 Atributos y operaciones de la clase Planeta.

Decidimos utilizar dos matrices porque el combustible y la peligrosidad son características diferentes de una conexión. Por ejemplo, una conexión puede necesitar poco combustible y aun así estar marcada como peligrosa.

Esta separación facilita las consultas durante la exploración. El algoritmo puede comprobar primero el tipo de conexión y después comparar el combustible de los destinos disponibles.

4.2.3 Registro de los planetas

Para incorporar un planeta al grafo implementamos la operación `agregarPlaneta`, que se presenta en la Figura 5.

Antes de registrarlo, la operación verifica que el nombre no esté vacío, que no exista otro planeta con el mismo nombre y que el factor temporal sea mayor o igual a uno.

Estas comprobaciones permiten mantener válidos los datos almacenados en el grafo. Cada nombre debe identificar un único planeta, y el factor temporal se utiliza posteriormente en el cálculo del combustible.

Si alguna condición no se cumple, la operación retorna falso y no agrega el planeta.

En caso contrario, incorpora el planeta al final del vector y amplía las dos matrices de adyacencia.

Para ampliar las matrices, se añade una posición a cada fila existente y después se crea una nueva fila con el tamaño correspondiente a la cantidad de planetas registrados.



Fig. 4 Estructuras y operaciones utilizadas para representar el grafo.

Las nuevas posiciones de la matriz de pesos comienzan en cero, porque el combustible todavía no se ha calculado. Las posiciones de la matriz de peligrosidad comienzan en falso y se modifican cuando se registran las conexiones peligrosas.

De esta forma, las posiciones del vector y de las matrices permanecen relacionadas y permiten identificar correctamente los extremos de cada conexión.

4.2.4 Almacenamiento de las conexiones

Las conexiones entre planetas son bidireccionales. Por esta razón, el combustible necesario para desplazarse entre dos planetas debe ser igual en ambos sentidos.

En la Figura 6 se muestran las operaciones `establecerPeso` y `marcarPeligrosa`.

La operación `establecerPeso` recibe las posiciones de dos planetas y el combustible calculado. Después registra ese valor en las dos posiciones correspondientes de la matriz de pesos.

Así, consultar una conexión desde el planeta de origen hacia el destino produce el mismo resultado que consultarla en sentido contrario.



Fig. 5 Registro de un planeta y ampliación de las matrices de adyacencia.

Por su parte, `marcarPeligrosa` permite identificar una conexión gravitacionalmente peligrosa. La operación registra el valor verdadero en ambas posiciones de la matriz de booleanos.

Esta implementación evita que las conexiones tengan información diferente según el sentido en que se consulten.

También se dispone de las operaciones `obtenerPeso` y `esPeligrosa`, que permiten recuperar el combustible almacenado y determinar si una conexión es peligrosa.

Estas consultas son utilizadas posteriormente por el algoritmo de exploración para seleccionar el siguiente destino de la nave.

4.2.5 Cálculo de los pesos del grafo

Una vez registrados los planetas y verificadas sus conexiones peligrosas, el sistema ejecuta la operación `calcularCombustible`.

Esta operación recorre los pares de planetas y calcula cuánto combustible se necesita para viajar entre ellos.

Como se observa en la Figura 7, el recorrido utiliza dos ciclos. El segundo comienza en la posición siguiente al primer planeta seleccionado, de manera que cada par de planetas diferentes se procesa una sola vez.

Primero se consultan las coordenadas de ambos planetas y se calcula la distancia euclidiana.

Después se obtiene el factor gravitacional utilizando los factores temporales y la constante λ.

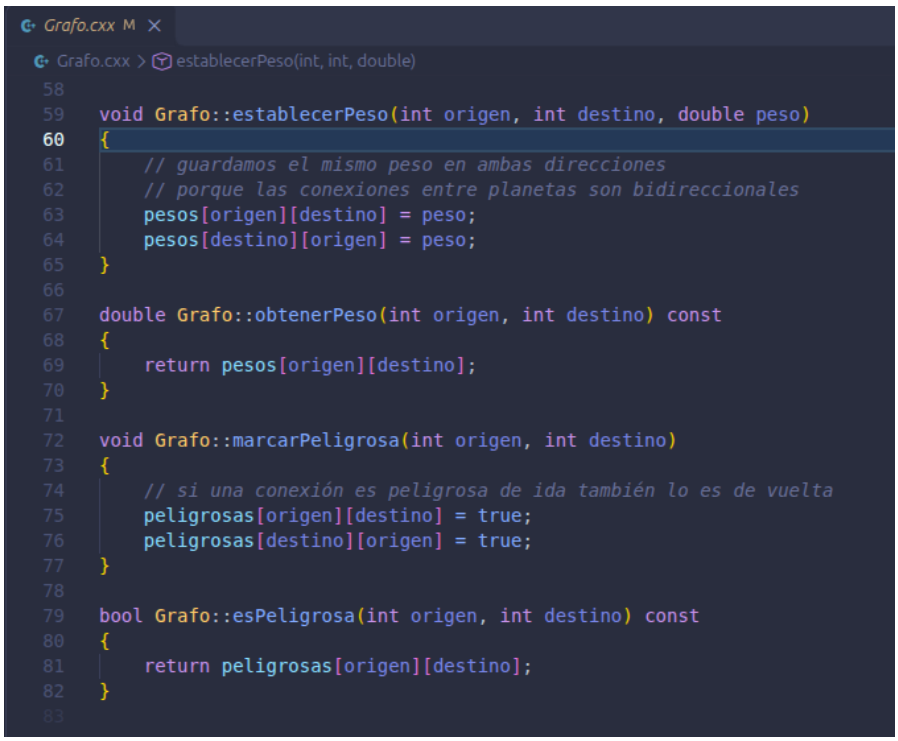


Fig. 6 Almacenamiento de pesos y conexiones peligrosas en ambos sentidos.

Finalmente, se multiplica la distancia por el factor gravitacional y por la constante k para obtener el combustible necesario para el desplazamiento.

El resultado se almacena mediante `grafo.establecerPeso`, que registra el mismo valor en ambos sentidos.

Esta operación es importante porque los pesos que utiliza el algoritmo de exploración no representan únicamente las distancias, sino el combustible calculado según las condiciones establecidas en el problema.

Con estas operaciones queda construido el grafo con la información de los planetas, sus pesos y las conexiones peligrosas.

El Sistema de Exploración Lázaro utilizará estos datos para determinar los desplazamientos de la nave durante la misión.

4.3 Descripción del Programa Principal

El programa principal organiza las operaciones necesarias para desarrollar la Misión Lázaro. Su funcionamiento comienza con la creación del Sistema de Exploración Lázaro y la lectura del nombre del archivo de entrada.

Después se ejecutan las operaciones de este TAD para cargar los planetas, calcular el combustible, realizar la exploración y generar los archivos con los resultados de la misión.

SistemaExploracionLazaro.cxx

SistemaExploracionLazaro.cxx > cargarPlanetas(const string &)

```cpp
void SistemaExploracionLazaro::calcularCombustible()
{
    // calculamos el combustible necesario para viajar entre cada par de planetas
    // solo recorremos la parte superior de la matriz porque las conexiones
    // son bidireccionales y Grafo ya guarda el mismo peso en ambos sentidos
    for (int i = 0; i < grafo.cantidadPlanetas(); i++)
    {
        for (int j = i + 1; j < grafo.cantidadPlanetas(); j++)
        {
            Planeta origen = grafo.obtenerPlaneta(i);
            Planeta destino = grafo.obtenerPlaneta(j);

            double diferenciaX = origen.obtenerX() - destino.obtenerX();
            double diferenciaY = origen.obtenerY() - destino.obtenerY();
            double diferenciaZ = origen.obtenerZ() - destino.obtenerZ();

            // usamos la distancia euclidiana entre las coordenadas
            // para conocer la separacion espacial de los dos planetas
            double distancia = sqrt(
                diferenciaX * diferenciaX +
                diferenciaY * diferenciaY +
                diferenciaZ * diferenciaZ);

            // el factor gravitacional ajusta el combustible teniendo en cuenta
            // los factores temporales de ambos planetas y el valor de lambda
            double factor = 1 + lambda *
                                ((origen.obtenerFactorTemporal() - 1) +
                                 (destino.obtenerFactorTemporal() - 1)) /
                                2;

            double combustible = k * distancia * factor;

            // guardamos el resultado como peso de la conexion
            // y Grafo se encarga de registrarlo en las dos direcciones
            grafo.establecerPeso(i, j, combustible);
        }
    }
```

Fig. 7 Cálculo del combustible necesario entre los pares de planetas.

Estas operaciones se ejecutan en ese orden porque cada una utiliza información obtenida en la anterior.

El recorrido no puede calcularse antes de construir el grafo y determinar los pesos de sus conexiones. De igual manera, los archivos de salida necesitan el recorrido obtenido durante la exploración.

El programa principal también comprueba el resultado de las operaciones que pueden fallar. Cuando alguna de ellas retorna falso, finaliza con un código diferente de cero. Si todas terminan correctamente, el programa finaliza con código cero.

4.3.1 Lectura del archivo de entrada

La operación `cargarPlanetas` recibe el nombre del archivo de entrada e intenta abrirlo.

Si no puede abrirse, informa el error y termina la carga sin éxito.

Cuando el archivo se abre correctamente, se leen la constante de combustible k, la constante de dilatación temporal λ y la cantidad de planetas.

El sistema comprueba que exista al menos un planeta, que k sea positiva y que λ no sea negativa.

Posteriormente se recorren los datos de cada planeta, que incluyen el nombre, las coordenadas espaciales, el factor temporal y las conexiones peligrosas declaradas.

Con la información leída se crea un objeto Planeta y se utilizan sus operaciones para asignarle los valores correspondientes.

El planeta se agrega al grafo mediante `agregarPlaneta`, que comprueba que su información sea válida.

Los nombres de las conexiones peligrosas se conservan temporalmente, porque al leer un planeta puede que todavía no se hayan registrado todos sus posibles destinos.

Después de cargar todos los planetas, se revisan las conexiones declaradas.

Cada planeta de destino debe existir en el grafo, ser diferente del origen y no estar repetido dentro de la lista. Además, una conexión peligrosa debe estar declarada en ambos sentidos.

Si se cumplen estas condiciones, se registran las conexiones peligrosas en la matriz correspondiente del grafo.

4.3.2 Construcción del grafo

Durante la carga, la clase Grafo incorpora los planetas en un vector y amplía las matrices que almacenan sus conexiones.

Una vez terminada esta etapa, el programa principal ejecuta la operación `calcularCombustible`.

Esta operación obtiene los datos de cada par de planetas y utiliza sus coordenadas y factores temporales para calcular el peso de la conexión.

Primero determina la distancia euclidiana. Después calcula el factor gravitacional y finalmente obtiene el combustible requerido.

Los resultados se almacenan en la matriz de pesos mediante la operación `establecerPeso`.

Como las conexiones son bidireccionales, los pesos quedan registrados en ambos sentidos.

Al terminar este procedimiento, el grafo conserva todos los datos necesarios para seleccionar los desplazamientos de la nave.

4.3.3 Ejecución de la exploración

Después de calcular el combustible, el programa principal llama a `realizarExploracion`.

Esta operación busca la Tierra dentro del grafo y la establece como el punto de partida de la nave.

Después prepara un vector para identificar los planetas visitados, limpia el recorrido y establece el combustible total en cero.

La exploración comienza desde la Tierra, que se agrega al recorrido y se marca como visitada.

Para seleccionar el siguiente destino, el algoritmo revisa los planetas que todavía no han sido visitados y las conexiones que salen del planeta actual.

Primero considera las conexiones seguras y elige la que necesita menos combustible. Si no existe una conexión segura disponible, considera las conexiones peligrosas.

Cuando existen dos opciones del mismo tipo y con igual combustible, se utiliza el nombre de los planetas para realizar el desempate alfabético.

Después de seleccionar un destino, se registra el desplazamiento, se suma el combustible utilizado, se marca el planeta como visitado y se actualiza la posición de la nave.

El procedimiento continúa hasta visitar todos los planetas registrados.

Finalmente, la nave regresa directamente a la Tierra y el sistema conserva el recorrido completo y el combustible total consumido.

Las condiciones utilizadas para escoger cada destino se explican con mayor detalle en el apartado 4.4.

4.3.4 Generación de los archivos de salida

Cuando la exploración termina correctamente, el programa principal ejecuta `generarDetalle` y `generarResumen`.

La primera operación crea el archivo `detalle_recorrido.txt`, que contiene la información de cada desplazamiento realizado por la nave.

Para obtener estos datos, el sistema recorre los planetas consecutivos del recorrido.

En cada desplazamiento identifica el origen y el destino, calcula la distancia y el factor gravitacional, y consulta el combustible y la peligrosidad de la conexión.

Después escribe la información del desplazamiento y actualiza el combustible acumulado.

Este procedimiento también incluye el regreso final a la Tierra.

La operación `generarResumen` crea el archivo `resumen.txt`, que presenta los resultados generales de la misión.

Para ello, el sistema utiliza el recorrido y calcula la cantidad de desplazamientos, la distancia total, las conexiones seguras y peligrosas utilizadas, y los movimientos de mayor y menor consumo.

El archivo también incluye la cantidad de planetas registrados y explorados, el recorrido completo y el combustible total de la misión.

Ambas operaciones verifican que exista un recorrido y que los archivos puedan crearse correctamente.

Además, después de escribir la información, comprueban que no se haya producido un error durante la generación de los archivos.

4.3.5 Pseudocódigo del programa principal

A continuación se presenta el procedimiento general utilizado por el programa principal.

El pseudocódigo muestra el orden de las operaciones y las principales decisiones que se toman durante la carga, la construcción del grafo, la exploración y la generación de los archivos de salida.

1. Iniciar el programa principal.

1.1. Crear un objeto del TAD Sistema de Exploración Lázaro.

1.2. Solicitar al usuario el nombre del archivo que contiene los datos de la misión.

1.3. Leer el nombre del archivo ingresado.

2. Cargar los planetas y las conexiones peligrosas de la misión.

2.1. Intentar abrir el archivo de entrada.

2.1.1. Si el archivo no puede abrirse, mostrar un mensaje de error y finalizar la carga.

2.1.2. Si el archivo se abre correctamente, continuar con la lectura.

2.2. Leer las constantes k y λ, y la cantidad de planetas.

2.3. Comprobar que la cantidad de planetas sea positiva, que k sea mayor que cero y que λ no sea negativa.

2.3.1. Si los datos iniciales son inválidos, informar el error y terminar la carga.

2.4. Preparar una colección temporal para guardar las listas de conexiones peligrosas.

2.5. Repetir los siguientes pasos por cada planeta indicado en el archivo.

2.5.1. Leer el nombre, las coordenadas, el factor temporal y la cantidad de conexiones peligrosas.

2.5.2. Comprobar que los datos se hayan leído correctamente y que la cantidad de conexiones peligrosas no sea negativa.

2.5.2.1. Si existe un error de lectura, informarlo y finalizar la carga.

2.5.3. Crear un objeto Planeta.

2.5.4. Asignar su nombre, sus coordenadas y su factor temporal.

2.5.5. Solicitar al TAD Grafo que agregue el planeta.

2.5.5.1. Comprobar que el nombre no esté vacío ni repetido y que el factor temporal sea mayor o igual a uno.

2.5.5.2. Si el planeta es inválido, informar el error y finalizar la carga.

2.5.5.3. Si es válido, incorporarlo al vector de planetas y ampliar las matrices.

2.5.6. Leer los nombres de las conexiones peligrosas declaradas para el planeta.

2.5.7. Guardar temporalmente esos nombres para verificarlos después de registrar todos los planetas.

2.6. Cerrar el archivo una vez leídos todos los planetas.

3. Verificar las conexiones peligrosas.

3.1. Recorrer las listas de conexiones peligrosas guardadas temporalmente.

3.2. Buscar en el grafo el planeta de destino indicado por cada nombre.

3.2.1. Si el destino no existe o coincide con el planeta de origen, informar el error y finalizar la carga.

3.3. Comprobar que el destino no aparezca repetido en la lista del origen.

3.3.1. Si aparece repetido, informar el error y finalizar la carga.

3.4. Comprobar que el planeta de destino también declare al origen como una conexión peligrosa.

3.4.1. Si la conexión no aparece en ambos sentidos, informar el error y finalizar la carga.

3.5. Repetir las verificaciones hasta revisar todas las conexiones.

3.6. Si todas son válidas, marcarlas como peligrosas en el grafo.

3.7. Regresar al programa principal con la carga completada.

4. Comprobar el resultado de la operación cargarPlanetas.

4.1. Si la carga falló, finalizar el programa con código uno.

4.2. Si la carga terminó correctamente, continuar con los pesos.

5. Calcular el combustible de las conexiones del grafo.

5.1. Recorrer cada par de planetas diferentes una sola vez.

5.2. Obtener sus coordenadas espaciales.

5.3. Calcular la distancia euclidiana entre los planetas.

5.4. Consultar los factores temporales y calcular el factor gravitacional.

5.5. Calcular el combustible requerido utilizando k, la distancia y el factor gravitacional.

5.6. Guardar el peso de la conexión en ambos sentidos.

5.7. Repetir hasta completar todos los pares de planetas.

5.8. Mostrar por consola la matriz de combustible y los planetas registrados.

6. Realizar la exploración mediante el algoritmo de Prim modificado.

6.1. Buscar la Tierra en el grafo.

6.1.1. Si no existe, informar el error y finalizar la exploración.

6.2. Preparar el vector de visitados, el recorrido y el combustible total.

6.3. Establecer Tierra como planeta actual, agregarla al recorrido y marcarla como visitada.

6.4. Mientras queden planetas pendientes, buscar un destino desde el planeta actual.

6.4.1. Revisar primero las conexiones seguras hacia los planetas no visitados.

6.4.1.1. Seleccionar la conexión segura que requiera menos combustible.

6.4.2. Si no existe una conexión segura, revisar las conexiones peligrosas.

6.4.2.1. Seleccionar la peligrosa de menor combustible.

6.4.3. Si varias opciones tienen el mismo combustible, seleccionar el destino cuyo nombre aparece primero alfabéticamente.

6.4.4. Si no existe un destino válido, informar el error y terminar la exploración.

6.4.5. Consultar el combustible y la peligrosidad del movimiento.

6.4.6. Mostrar el desplazamiento y sumar su combustible al total.

6.4.7. Agregar el destino al recorrido y marcarlo como visitado.

6.4.8. Actualizar el planeta actual con el destino seleccionado.

6.5. Cuando se hayan visitado todos los planetas, regresar directamente a Tierra.

6.5.1. Si la nave está en otro planeta, consultar el combustible y la peligrosidad del regreso.

6.5.2. Registrar el desplazamiento aunque la conexión sea peligrosa.

6.5.3. Sumar el combustible del regreso y agregar Tierra al recorrido.

6.6. Conservar el recorrido y el combustible total en el sistema.

7. Comprobar el resultado de realizarExploracion.

7.1. Si la exploración falló, finalizar el programa con código uno.

7.2. Si terminó correctamente, continuar con los archivos de salida.

8. Generar el archivo de detalle del recorrido.

8.1. Comprobar que exista un recorrido y abrir detalle_recorrido.txt.

8.1.1. Si no hay recorrido o el archivo no puede crearse, informar el error y finalizar la operación.

8.2. Preparar el formato con dos decimales y escribir el encabezado.

8.3. Recorrer los pares consecutivos de planetas del recorrido.

8.3.1. Obtener el origen y el destino del desplazamiento.

8.3.2. Calcular la distancia euclidiana y el factor gravitacional.

8.3.3. Consultar el combustible y la peligrosidad de la conexión.

8.3.4. Actualizar el combustible acumulado.

8.3.5. Escribir el origen, el destino, la distancia, el factor gravitacional, el combustible, el tipo de conexión y el consumo acumulado.

8.4. Cerrar el archivo y comprobar que no haya ocurrido un error de escritura.

9. Comprobar el resultado de generarDetalle.

9.1. Si la operación falló, finalizar el programa con código uno.

10. Generar el archivo de resumen de la misión.

10.1. Comprobar que exista un recorrido y abrir resumen.txt.

10.1.1. Si no hay recorrido o el archivo no puede crearse, informar el error y finalizar la operación.

10.2. Consultar la cantidad de planetas y calcular el número de desplazamientos.

10.3. Preparar los acumuladores de distancia y los contadores de conexiones.

10.4. Recorrer los desplazamientos registrados durante la misión.

10.4.1. Calcular y acumular la distancia de cada desplazamiento.

10.4.2. Consultar su combustible y su peligrosidad.

10.4.3. Actualizar los contadores de conexiones seguras y peligrosas utilizadas.

10.4.4. Identificar los movimientos de mayor y menor consumo.

10.5. Escribir los planetas registrados y explorados, el recorrido y la cantidad de desplazamientos.

10.6. Escribir la distancia total, el combustible total y la cantidad de conexiones de cada tipo.

10.7. Escribir los desplazamientos de mayor y menor consumo, si se realizó alguno.

10.8. Escribir el resultado de la misión y la confirmación del regreso a Tierra.

10.9. Cerrar el archivo y comprobar que su escritura haya terminado correctamente.

11. Comprobar el resultado de generarResumen.

11.1. Si la operación falló, finalizar el programa con código uno.

12. Finalizar el programa correctamente.

12.1. Retornar código cero cuando todas las operaciones hayan terminado sin errores.

4.4 Descripción del algoritmo modificado

Para determinar el recorrido de la nave utilizamos una modificación del algoritmo de Prim. En la Misión Lázaro, la nave debe comenzar en la Tierra, visitar los demás planetas una sola vez y regresar al punto de partida.

Durante el recorrido se debe considerar el combustible necesario para cada desplazamiento y la existencia de conexiones gravitacionales peligrosas.

El algoritmo de Prim tradicional construye un árbol de expansión mínima. Para ello, selecciona conexiones que unen los vértices ya incluidos en el árbol con los vértices que todavía no forman parte de este.

Sin embargo, en nuestro problema necesitamos determinar el recorrido de una nave. Cada desplazamiento debe comenzar desde el planeta donde se encuentra actualmente.

Por esta razón, adaptamos el procedimiento para buscar el siguiente destino únicamente entre las conexiones que salen del planeta actual, en lugar de considerar todas las conexiones que salen del conjunto de planetas visitados.

Además, incorporamos una prioridad para las conexiones seguras, un criterio de desempate y el regreso obligatorio a la Tierra.

4.4.1 Información utilizada por el algoritmo

Antes de comenzar la exploración, el programa ya ha cargado los planetas y calculado el combustible necesario para desplazarse entre ellos.

Por lo tanto, el algoritmo utiliza la información almacenada en el TAD Grafo, principalmente la matriz de pesos y la matriz de conexiones peligrosas.

La matriz de pesos contiene el combustible necesario para viajar entre cada par de planetas.

Este combustible ya fue calculado mediante la distancia euclidiana, los factores temporales y las constantes k y λ, como se explicó en el apartado 4.1.

Durante la exploración no se calculan nuevamente estos valores. El algoritmo los consulta mediante la operación `obtenerPeso`.

Por otra parte, la operación `esPeligrosa` permite comprobar si una conexión está marcada como gravitacionalmente peligrosa.

También utilizamos un vector de visitados, que indica cuáles planetas ya fueron explorados y cuáles permanecen pendientes.

El sistema mantiene el planeta actual, el recorrido realizado y el combustible total consumido durante la misión.

4.4.2 Inicio de la exploración

La exploración comienza buscando el planeta llamado Tierra dentro del grafo.

Si la Tierra no se encuentra registrada, no es posible comenzar la misión y la operación termina sin éxito.

Cuando se encuentra, se utiliza su posición como punto de partida.

Después se prepara el vector de planetas visitados, inicialmente con todas sus posiciones en cero.

El recorrido se limpia y el combustible total se establece en cero para comenzar una nueva exploración.

La Tierra se agrega al inicio del recorrido, se marca como visitada y se establece como planeta actual.

A partir de ese momento comienza la búsqueda de los demás planetas.

4.4.3 Prioridad de las conexiones seguras

Una de las modificaciones principales consiste en dar prioridad a las conexiones seguras durante la selección del siguiente planeta.

En cada desplazamiento se revisan primero las conexiones que salen del planeta actual hacia planetas que todavía no han sido visitados.

Entre estas conexiones se consideran inicialmente las que no están marcadas como peligrosas.

Si existe al menos una conexión segura, se selecciona el destino que necesita menos combustible.

Esto significa que la nave puede seleccionar una conexión segura de mayor combustible antes que una conexión peligrosa de menor combustible.

La seguridad de la conexión tiene prioridad sobre el consumo cuando se comparan conexiones de diferentes tipos.

Solamente cuando no existe ninguna conexión segura hacia un planeta pendiente se consideran las conexiones peligrosas.

En ese caso, se selecciona la conexión peligrosa que necesita menos combustible desde el planeta actual.

De esta manera, las conexiones peligrosas se utilizan cuando son necesarias para continuar la exploración.

4.4.4 Selección del destino y desempate

Después de identificar las conexiones del tipo permitido, el algoritmo compara el combustible necesario para cada posible desplazamiento.

El destino seleccionado será el que tenga el menor peso entre las conexiones del mismo tipo.

Si se encuentra otra conexión con un combustible menor, se reemplaza el destino que estaba seleccionado.

Cuando dos conexiones del mismo tipo necesitan exactamente la misma cantidad de combustible, utilizamos el nombre de los planetas para decidir.

En ese caso, se selecciona el planeta cuyo nombre aparece primero alfabéticamente.

Este criterio permite determinar un destino cuando el combustible y el tipo de conexión no permiten distinguir entre las opciones.

Si después de revisar las conexiones seguras y peligrosas no existe un destino válido, la operación termina sin éxito.

4.4.5 Actualización del recorrido

Una vez seleccionado el siguiente planeta, el sistema consulta el combustible necesario para realizar el desplazamiento.

También comprueba si la conexión seleccionada es segura o peligrosa.

Esta información se muestra por consola, indicando el origen, el destino, el combustible y el tipo de conexión.

El combustible utilizado se suma al total acumulado de la misión.

Después, el planeta elegido se agrega al vector que almacena el recorrido.

También se marca como visitado para evitar que sea seleccionado nuevamente durante la exploración.

Finalmente, el destino se convierte en el nuevo planeta actual.

La siguiente búsqueda comienza desde esta nueva posición, no desde cualquiera de los planetas visitados anteriormente.

El procedimiento se repite hasta que todos los planetas hayan sido explorados.

4.4.6 Regreso obligatorio a la Tierra

Cuando todos los planetas han sido visitados, la nave debe regresar directamente a la Tierra.

Para realizar este desplazamiento, el sistema consulta el combustible necesario desde el último planeta visitado hasta el punto de partida.

También comprueba la peligrosidad de la conexión de regreso.

En esta última etapa no se busca otro planeta seguro, porque todos los destinos de exploración ya fueron visitados y la Tierra es el destino obligatorio.

Por esta razón, el regreso se realiza incluso cuando la conexión está marcada como peligrosa.

El combustible del regreso se suma al total consumido y la Tierra se agrega al final del recorrido.

Si solamente existe la Tierra, la nave ya se encuentra en el punto de partida y no necesita realizar otro desplazamiento.

4.4.7 Pseudocódigo de Prim modificado

El siguiente pseudocódigo presenta el procedimiento utilizado para seleccionar cada destino.

Se incluyen las comprobaciones de seguridad, la comparación del combustible, el control de los planetas visitados y el regreso final a la Tierra.

1. Iniciar la exploración utilizando el grafo con los planetas y sus conexiones registradas.

1.1. Buscar el planeta Tierra dentro del grafo.

1.1.1. Si Tierra no existe, informar el error y terminar la exploración sin éxito.

1.1.2. Si existe, guardar su posición como punto de partida.

2. Preparar la información necesaria para desarrollar el recorrido.

2.1. Obtener la cantidad de planetas registrados.

2.2. Crear un vector de visitados con una posición por planeta, inicialmente en cero.

2.3. Limpiar el recorrido anterior y establecer el combustible total en cero.

2.4. Establecer Tierra como planeta actual.

2.5. Agregar Tierra al recorrido y marcarla como visitada.

3. Comenzar la exploración de los demás planetas.

3.1. Mientras falten planetas por visitar, seleccionar un nuevo destino.

3.1.1. Iniciar la búsqueda sin un destino seleccionado.

3.1.2. Buscar primero las conexiones seguras que salen del planeta actual.

3.1.2.1. Recorrer los planetas registrados en el grafo.

3.1.2.2. Comprobar que cada planeta no haya sido visitado.

3.1.2.2.1. Si ya fue visitado, descartarlo como destino.

3.1.2.3. Comprobar que la conexión desde el planeta actual sea segura.

3.1.2.3.1. Si es peligrosa, no considerarla durante la búsqueda de seguras.

3.1.2.4. Si el planeta no ha sido visitado y la conexión es segura, considerarlo como posible destino.

3.1.2.5. Si todavía no hay candidato, seleccionar el primer destino seguro encontrado.

3.1.2.6. Si ya existe un candidato, comparar el combustible de ambas conexiones.

3.1.2.6.1. Si la nueva conexión necesita menos combustible, reemplazar el destino elegido.

3.1.2.6.2. Si necesita más combustible, conservar el destino anterior.

3.1.2.6.3. Si ambas conexiones requieren el mismo combustible, comparar los nombres de los planetas.

3.1.2.6.4. Conservar el destino cuyo nombre aparece primero alfabéticamente.

3.1.3. Comprobar si se encontró un destino seguro.

3.1.3.1. Si existe, conservarlo como siguiente destino.

3.1.3.2. Si no existe, comenzar la búsqueda de conexiones peligrosas.

3.1.4. Revisar las conexiones peligrosas desde el planeta actual.

3.1.4.1. Recorrer nuevamente los planetas registrados.

3.1.4.2. Considerar únicamente los planetas no visitados cuya conexión desde el planeta actual sea peligrosa.

3.1.4.3. Si todavía no existe un candidato, seleccionar el primer destino peligroso válido.

3.1.4.4. Comparar el combustible de las demás conexiones peligrosas disponibles.

3.1.4.4.1. Si una conexión necesita menos combustible, actualizar el destino.

3.1.4.4.2. Si hay empate, seleccionar el planeta cuyo nombre aparece primero alfabéticamente.

3.1.5. Comprobar si existe un destino después de revisar las conexiones.

3.1.5.1. Si no existe un destino válido, informar el error y terminar la exploración sin éxito.

3.1.5.2. Si existe, continuar con el desplazamiento.

4. Registrar el desplazamiento hacia el planeta seleccionado.

4.1. Consultar el combustible desde el planeta actual hasta el destino elegido.

4.2. Consultar si la conexión es segura o peligrosa.

4.3. Mostrar por consola el origen, el destino, el combustible y el tipo de conexión.

4.4. Sumar el combustible del desplazamiento al total de la misión.

4.5. Agregar el planeta elegido al final del recorrido.

4.6. Marcar el planeta elegido como visitado.

4.7. Actualizar la posición actual de la nave con el destino seleccionado.

4.8. Regresar al paso 3 mientras queden planetas sin visitar.

5. Realizar el regreso obligatorio a Tierra.

5.1. Comprobar si la nave se encuentra en un planeta diferente de Tierra.

5.1.1. Si ya está en Tierra, no realizar otro desplazamiento.

5.1.2. Si está en otro planeta, preparar el regreso directo.

5.2. Consultar el combustible y la peligrosidad de la conexión de regreso.

5.3. Registrar el regreso incluso cuando la conexión sea peligrosa.

5.4. Mostrar por consola el desplazamiento hacia Tierra.

5.5. Sumar el combustible del regreso al total.

5.6. Agregar Tierra al final del recorrido.

6. Finalizar la exploración.

6.1. Conservar el recorrido y el combustible total en el sistema.

6.2. Terminar correctamente para que el programa principal pueda generar los archivos de salida.

4.4.8 Resultado de la modificación

La diferencia principal frente al algoritmo de Prim tradicional es que el siguiente destino se selecciona desde el planeta actual de la nave.

En Prim tradicional, una nueva arista puede salir de cualquiera de los vértices que ya pertenecen al árbol.

En nuestra adaptación, la nave solamente puede partir del planeta donde se encuentra.

Además, las conexiones seguras tienen prioridad sobre las peligrosas, los planetas visitados no vuelven a seleccionarse y se incorpora el regreso obligatorio a Tierra.

Estas condiciones permiten construir un recorrido que cumple las reglas de exploración establecidas.

Sin embargo, la selección se realiza tomando una decisión en cada paso. Por lo tanto, el procedimiento no garantiza que el combustible total sea el menor entre todos los recorridos posibles.

El recorrido obtenido y el combustible acumulado quedan disponibles para generar los archivos de detalle y resumen de la misión.

4.5 Implementación del algoritmo modificado en C++

El algoritmo de Prim modificado se implementó en la operación `realizarExploracion` del TAD Sistema de Exploración Lázaro.

Esta operación utiliza el grafo construido anteriormente para determinar el orden en que la nave visita los planetas y regresa a la Tierra.

Durante la ejecución se identifican los planetas pendientes, se consultan los pesos y la peligrosidad de las conexiones, se selecciona cada destino y se registra el combustible consumido.

A continuación se presentan los principales fragmentos del código implementado y se explica cómo se relacionan con el procedimiento descrito en el apartado 4.4.

4.5.1 Identificación de los planetas pendientes

Al iniciar la operación, el programa busca la Tierra en el grafo mediante `grafo.buscarPlaneta`.

Si no la encuentra, informa el error y termina la exploración sin éxito.

Después obtiene la cantidad de planetas y crea el vector `visitados`, con todas sus posiciones inicialmente en cero.

Cada posición del vector corresponde a un planeta del grafo. El valor cero indica que todavía no ha sido visitado y el valor uno indica que ya forma parte del recorrido.

Como se muestra en la Figura 8, la Tierra se agrega al recorrido y se marca como visitada antes de comenzar el ciclo.

El ciclo se ejecuta para visitar los demás planetas, sin volver a considerar la Tierra como un destino pendiente.

4.5.2 Priorización de las conexiones seguras

Para seleccionar el siguiente planeta, el programa recorre las conexiones que salen del planeta actual.

En la Figura 9 se observa que la variable `tipo` comienza en cero, correspondiente a las conexiones seguras.

El ciclo permite considerar posteriormente el valor uno, correspondiente a las conexiones peligrosas.

Sin embargo, este segundo tipo solamente se revisa si todavía no se ha encontrado un destino.

Dentro del ciclo se comprueba que el planeta no haya sido visitado mediante la condición `visitados[i] == 0`.

También se compara el resultado de `grafo.esPeligrosa` con el tipo de conexión que se está buscando.

De esta manera, el algoritmo considera primero los destinos pendientes conectados mediante aristas seguras.

Únicamente cuando no existe ninguna conexión segura disponible desde el planeta actual se consideran las peligrosas.

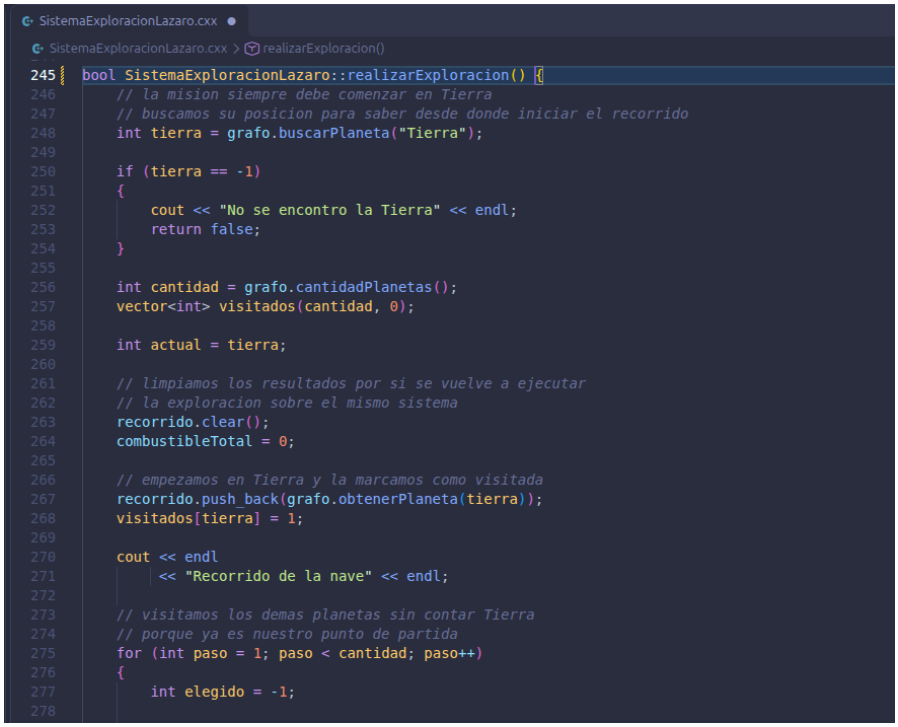


Fig. 8 Inicialización del recorrido e identificación de los planetas pendientes.

4.5.3 Selección del planeta de menor combustible

Una vez identificadas las conexiones del tipo que se está revisando, el programa compara el combustible necesario para llegar a cada destino.

En la Figura 10 se muestra la utilización de la operación `grafo.obtenerPeso` para consultar los pesos de las conexiones.

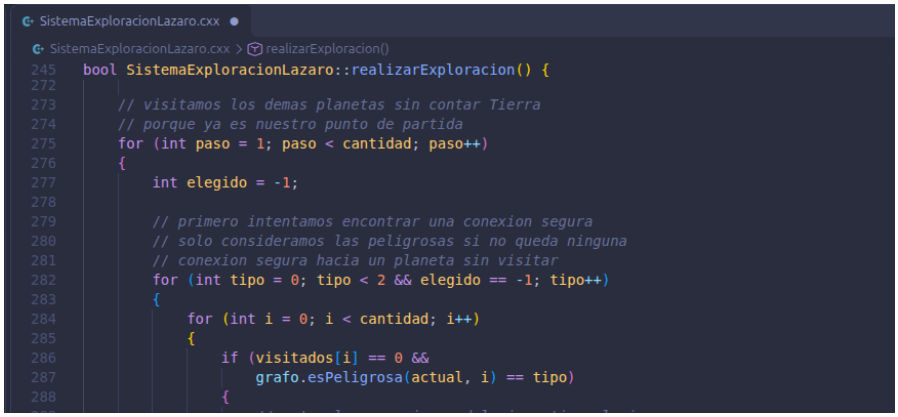


Fig. 9 Verificación de los planetas pendientes y prioridad de las conexiones seguras.

Es importante aclarar que estos valores ya fueron calculados por `calcularCombustible` antes de comenzar la exploración.

Por eso, dentro de `realizarExploracion` no se vuelve a calcular la distancia ni el factor gravitacional.

El algoritmo consulta el peso almacenado y lo compara con el del destino seleccionado hasta ese momento.

Si encuentra una conexión que necesita menos combustible, actualiza la posición del planeta elegido.

Cuando dos conexiones del mismo tipo requieren exactamente el mismo combustible, compara los nombres de los planetas y selecciona el que aparece primero alfabéticamente.

Si no existe ningún destino válido después de revisar ambos tipos de conexión, la operación retorna falso.

SistemaExploracionLazaro.cxx

SistemaExploracionLazaro.cxx > realizarExploracion()

```
245  bool SistemaExploracionLazaro::realizarExploracion() {
275      for (int paso = 1; paso < cantidad; paso++)
282          for (int tipo = 0; tipo < 2 && elegido == -1; tipo++)
284              for (int i = 0; i < cantidad; i++)
286                  if (visitados[i] == 0 &&
287                      grafo.esPeligrosa(actual, i) == tipo)
288                  {
289                      // entre las conexiones del mismo tipo elegimos
290                      // la que necesita menos combustible
291                      if (elegido == -1 ||
292                          grafo.obtenerPeso(actual, i) <
293                              grafo.obtenerPeso(actual, elegido))
294                      {
295                          elegido = i;
296                      }
297                      else if (grafo.obtenerPeso(actual, i) ==
298                                   grafo.obtenerPeso(actual, elegido) &&
299                               grafo.obtenerPlaneta(i).obtenerNombre() <
300                                   grafo.obtenerPlaneta(elegido).obtenerNombre())
301                      {
302                          // si hay empate en combustible usamos el nombre
303                          // alfabeticamente menor para decidir el destino
304                          elegido = i;
305                      }
306                  }
307              }
308          }
309
310          if (elegido == -1)
311          {
312              cout << "No se encontro otro planeta" << endl;
313              return false;
314          }
315
```

Fig. 10 Comparación de los pesos y selección del siguiente planeta.

4.5.4 Registro de los desplazamientos

Después de seleccionar el siguiente destino, el programa consulta el combustible y la peligrosidad de la conexión elegida.

Como se observa en la Figura 11, se obtiene el combustible mediante `grafo.obtenerPeso` y se consulta el tipo de conexión mediante `grafo.esPeligrosa`.

A continuación se muestra por consola el nombre del planeta de origen, el destino, el combustible necesario y si la conexión utilizada es segura o peligrosa.

El combustible del desplazamiento se suma a la variable `combustibleTotal`.

Esta variable conserva el consumo acumulado de la misión, incluyendo posteriormente el regreso a Tierra.

Además, el planeta seleccionado se agrega al vector `recorrido`, que conserva el orden real de las visitas.

Este recorrido se utiliza posteriormente para generar los archivos de detalle y resumen.

SistemaExploracionLazaro.cxx

SistemaExploracionLazaro.cxx > realizarExploracion()

```
245  bool SistemaExploracionLazaro::realizarExploracion() {
275      for (int paso = 1; paso < cantidad; paso++)
316          double combustible = grafo.obtenerPeso(actual, elegido);
317          bool peligrosa = grafo.esPeligrosa(actual, elegido);
318
319          cout << grafo.obtenerPlaneta(actual).obtenerNombre()
320               << " -> " << grafo.obtenerPlaneta(elegido).obtenerNombre()
321               << " | combustible: " << combustible << " | ";
322
323          if (peligrosa)
324          {
325              cout << "peligrosa" << endl;
326          }
327          else
328          {
329              cout << "segura" << endl;
330          }
331
332          combustibleTotal += combustible;
333
334          // guardamos el planeta elegido en el orden real de visita
335          // este recorrido tambien nos servira despues para generar
336          // los archivos de detalle y resumen de la mision
337          recorrido.push_back(grafo.obtenerPlaneta(elegido));
338
```

Fig. 11 Consulta del combustible y registro de los desplazamientos realizados.

4.5.5 Actualización de los planetas visitados

Después de registrar el desplazamiento, el programa marca el planeta seleccionado como visitado.

En la Figura 12 se observa esta actualización mediante la instrucción `visitados[elegido] = 1`.

Esta asignación impide que el planeta vuelva a considerarse como un destino pendiente durante los siguientes desplazamientos.

También se actualiza la posición actual de la nave mediante `actual = elegido`.

De esta manera, la siguiente búsqueda comienza desde el último planeta visitado, que es donde se encuentra la nave después del movimiento.

Estas operaciones permiten mantener la secuencia del recorrido y evitar que un planeta se seleccione más de una vez durante la exploración.

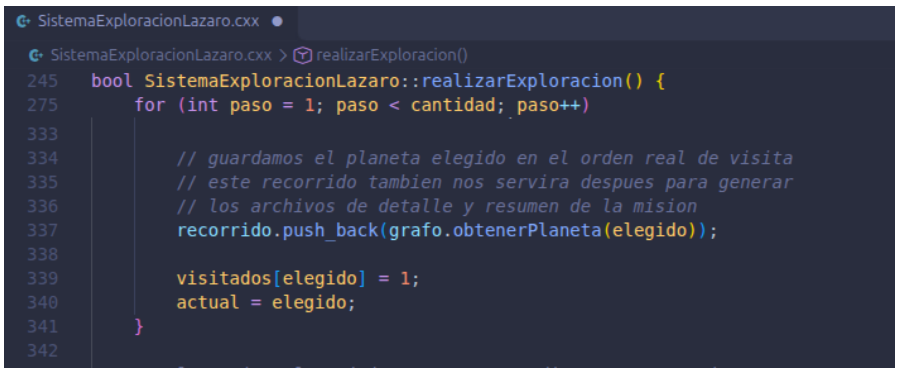


Fig. 12 Actualización de los planetas visitados y de la posición actual de la nave.

4.5.6 Regreso final a la Tierra

Cuando termina el ciclo de exploración, el programa comprueba si la nave se encuentra en un planeta diferente de Tierra.

Como se muestra en la Figura 13, esta comprobación se realiza mediante la condición `actual != tierra`.

Si la nave se encuentra en otro planeta, se consulta el combustible necesario para regresar directamente a la Tierra.

También se comprueba si la conexión de regreso es segura o peligrosa.

En este caso no se realiza una nueva búsqueda de destinos, porque todos los planetas ya fueron visitados.

El regreso se registra incluso cuando la conexión final es peligrosa.

El combustible necesario se suma al total acumulado y la Tierra se agrega al final del recorrido.

Esto permite conservar el desplazamiento de regreso como parte de la misión y utilizarlo posteriormente en los archivos de salida.

Con esta implementación, la operación `realizarExploracion` utiliza las estructuras del grafo para seleccionar cada destino según el tipo de conexión y el combustible necesario.

El vector de visitados permite controlar los planetas pendientes, mientras que el vector de recorrido conserva la secuencia de desplazamientos.

Finalmente, el regreso a Tierra completa la misión y el combustible total queda almacenado para generar los archivos con los resultados.

4.6 Plan de pruebas y validación de resultados

Para verificar el funcionamiento de la Misión Lázaro seguimos el formato de plan de pruebas presentado en el Taller 0. Para cada operación principal se definieron casos con diferentes datos de entrada, se calcularon sus resultados esperados sin utilizar las salidas del programa y después se compararon con los resultados obtenidos durante las ejecuciones. Las tablas incluyen la descripción de cada caso, los valores de entrada, el resultado esperado y el obtenido. Después de cada plan presentamos una selección de pantallazos de ejecución, identificados por el caso correspondiente.



Fig. 13 Registro del regreso obligatorio a la Tierra.

Los tres primeros casos utilizan los archivos oficiales del taller. Los demás se prepararon para comprobar situaciones que no quedan cubiertas por esas entradas, como la necesidad de utilizar una conexión peligrosa, los empates y las validaciones de datos inválidos.

4.6.1 Compilación

Primero compilamos y enlazamos el programa con `g++` y el estándar C++17. Siguiendo la indicación del profesor, se compilaron los archivos `.cxx`, sin incluir directamente los encabezados `.h`. El comando terminó con código de salida cero, como se observa en la Figura 14.



Fig. 14 Compilación del programa con `g++` y C++17.

4.6.2 Archivos de entrada oficiales

Antes de comparar las salidas identificamos las entradas que corresponden a las pruebas 01, 02 y 03. Estas contienen los nombres y coordenadas de los planetas, las constantes, los factores temporales y las conexiones peligrosas. La Tabla 2 relaciona cada archivo con su captura, para dejar constancia de los datos utilizados.

Table 2 Archivos oficiales utilizados en las pruebas.

Descripción del caso	**Evidencia de entrada**
P01: caso básico	
P02: dilatación temporal	
P03: conexiones peligrosas	

4.6.3 Plan de pruebas: cálculo del combustible

Para comprobar esta operación utilizamos la distancia euclidiana y el factor gravitacional de los planetas. El combustible esperado se calculó con $F_{ij} = k\, d_{ij}\, G_{ij}$ y se comparó con los pesos y acumulados que presenta el programa. La Tabla 3 conserva las cuatro columnas del plan de pruebas utilizado por el profesor.

En la prueba 01 todos los factores temporales son uno, por lo que $G = 1$. El combustible coincide con la distancia de cada trayecto y la suma manual es $2 + 3 + 4 + 9 = 18$. En la prueba 02, cuando uno de los extremos es Miller, se obtiene:

$$G = 1 + 0.5 \left(\frac{(8-1) + (1-1)}{2} \right) = 2.75.$$

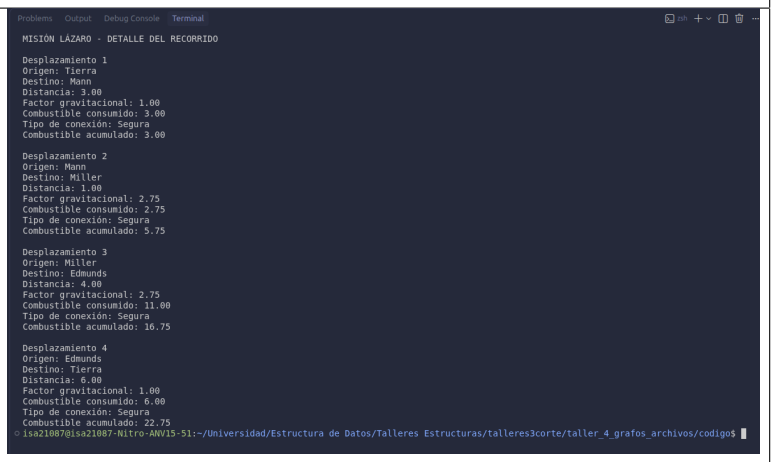
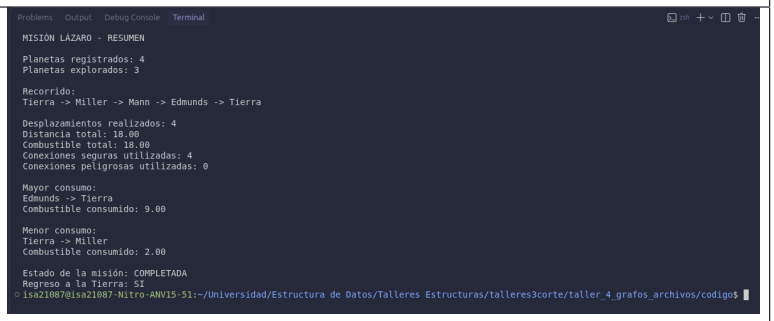
Por ejemplo, de Mann a Miller el consumo es $1 \cdot 2.75 = 2.75$; de Miller a Edmunds es $4 \cdot 2.75 = 11$. Sumados los cuatro movimientos, el total esperado es $3 + 2.75 + 11 + 6 = 22.75$. Los detalles de la segunda prueba se conservan en la evidencia de la Tabla 4.

Los cuatro casos de esta operación coincidieron con el cálculo manual. En las pruebas 03 y 04 el carácter peligroso de algunas conexiones no alteró el peso calculado de las aristas; se utilizó para decidir cuáles podían seleccionarse primero.

Table 3 Plan de pruebas: cálculo del combustible.

Descripción del caso	**Valores de entrada**	**Resultado esperado**	**Resultado obtenido**
P01: factores temporales iguales a uno	$k = 1$, $\lambda = 0.2$, $t = 1$ en todos los planetas	Consumos $2, 3, 4, 9$; total 18.00	Consumos $2, 3, 4, 9$; total 18.00
P02: factor temporal distinto	$k = 1$, $\lambda = 0.5$; Miller $t = 8$, los demás $t = 1$	Con Miller, $G = 2.75$; consumos $3, 2.75, 11, 6$; total 22.75	$G = 2.75$; consumos $3, 2.75, 11, 6$; total 22.75
P03: conexiones peligrosas	$k = 1$, $\lambda = 0.2$, $t = 1$; dos conexiones marcadas peligrosas	Consumos $3, 3, 5, 1$; total 12.00	Consumos $3, 3, 5, 1$; total 12.00
P04: conexión peligrosa necesaria	Tierra(0), Miller(1), Mann(3); $t = 1$	Consumos $1, 2, 3$; total 6.00	Consumos $1, 2, 3$; total 6.00

Table 4 Evidencias del cálculo del combustible.

Descripción del caso	**Evidencia de ejecución**
P02: detalle de factores y consumo	
P01: resumen del consumo acumulado	

4.6.4 Plan de pruebas: selección del recorrido

En esta operación comprobamos que la nave comience en Tierra, visite los otros planetas, dé prioridad a las conexiones seguras, compare el combustible dentro del tipo disponible y regrese a Tierra.

Table 5 Plan de pruebas: exploración mediante Prim modificado.

Descripción del caso	**Valores de entrada**	**Resultado esperado**	**Resultado obtenido**
P01: recorrido seguro	`prueba_01_basica.txt`; cuatro planetas sin peligros	Tierra–Miller–Mann–Edmunds–Tierra; 4 seguras	Mismo recorrido; 4 seguras
P02: dilatación temporal	`prueba_02_dilatacion.txt`; Miller con $t = 8$	Tierra–Mann–Miller–Edmunds–Tierra; 4 seguras	Mismo recorrido; 4 seguras
P03: seguridad prioritaria y regreso peligroso	Tierra–Miller y Miller–Mann peligrosas	Tierra–Mann–Edmunds–Miller–Tierra; 3 seguras y 1 peligrosa	Mismo recorrido; 3 seguras y 1 peligrosa
P04: peligrosa durante la exploración	No hay conexión segura desde Tierra a un planeta pendiente	Tierra–Miller–Mann–Tierra; 1 segura y 2 peligrosas	Mismo recorrido; 1 segura y 2 peligrosas

En P03, viajar de Tierra a Miller cuesta una unidad, pero la conexión es peligrosa. El programa escoge Tierra–Mann, de costo tres, porque es segura. Después visita Edmunds y Miller; el último movimiento Miller–Tierra es peligroso, pero debe realizarse porque el regreso es obligatorio. P04 permite comprobar otro comportamiento: cuando no queda una conexión segura al comenzar la exploración, sí se selecciona una peligrosa para continuar.

Los cuatro recorridos observados coincidieron con los esperados para sus respectivas entradas. Las Tablas 6 y 7 permiten comprobar el orden de visita y la selección de conexiones seguras y peligrosas.

Table 6 Evidencia de exploración: pruebas P01 y P02.

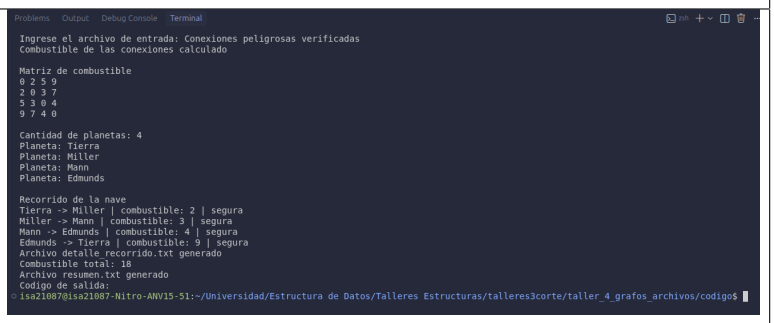
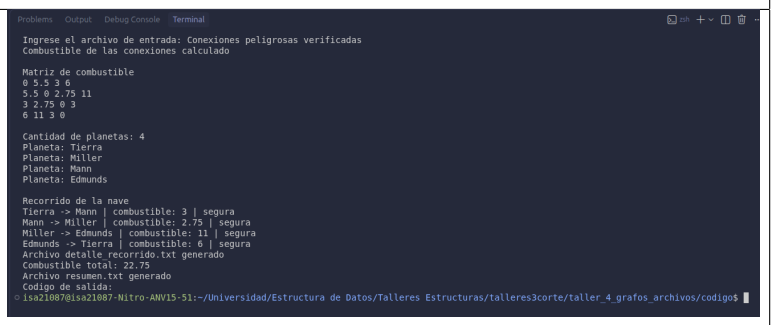
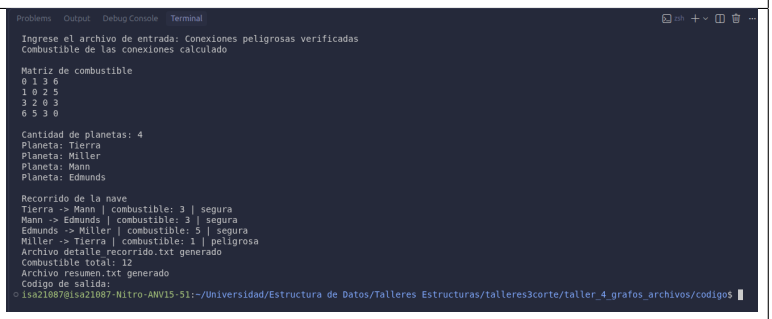
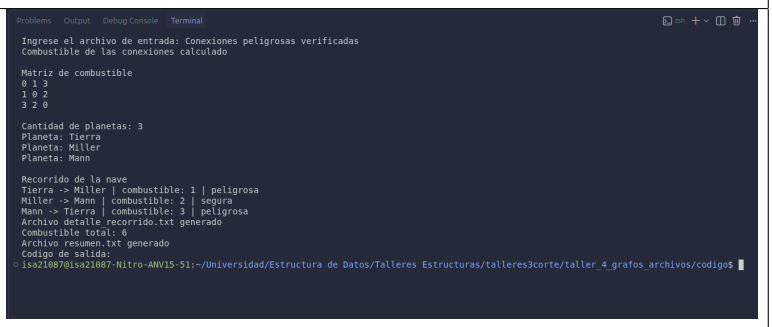
Descripción del caso	**Evidencia de ejecución**
P01: recorrido seguro	
P02: dilatación temporal	

Table 7 Evidencia de exploración: pruebas P03 y P04.

Descripción del caso	**Evidencia de ejecución**
P03: prioridad segura y regreso peligroso	
P04: conexión peligrosa necesaria	

4.6.5 Plan de pruebas: casos especiales

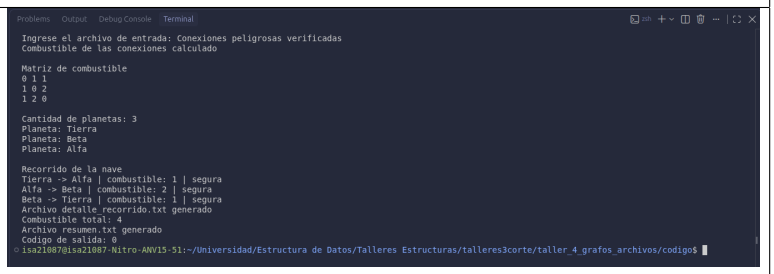
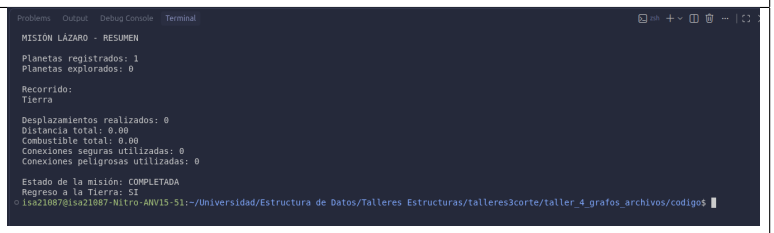
También probamos dos condiciones que no aparecen en los tres archivos oficiales: un empate de combustible y una entrada que contiene solamente la Tierra.

Table 8 Plan de pruebas: decisiones especiales del recorrido.

Descripción del caso	**Valores de entrada**	**Resultado esperado**	**Resultado obtenido**
P12: empate de combustible	Tierra(0), Beta(1), Alfa(−1); $t = 1$	Elegir Alfa primero; Tierra–Alfa–Beta–Tierra; 4.00	Alfa primero; mismo recorrido; 4.00
P13: solo Tierra	Un único planeta llamado Tierra, $t = 1$	Tierra; 0 desplazamientos; 0.00	Tierra; 0 desplazamientos; 0.00

En P12, ambos destinos requieren una unidad desde Tierra. El código utiliza el nombre para desempatar y selecciona Alfa, que aparece primero alfabéticamente. En P13 la misión no requiere ningún desplazamiento y el combustible total se mantiene en cero.

Table 9 Evidencias de las situaciones especiales.

Descripción del caso	**Evidencia de ejecución**
P12: desempate alfabético	
P13: misión con solo Tierra	

4.6.6 Plan de pruebas: validación de la entrada

La carga de datos también debe reconocer entradas inválidas. Para cada archivo comprobamos si el programa muestra el error correspondiente y termina con código de salida uno. Ese resultado se considera correcto para la validación, porque el programa no debe completar una misión con datos incorrectos.

Table 10 Plan de pruebas: archivos, constantes y planetas inválidos.

Descripción del caso	**Valores de entrada**	**Resultado esperado**	**Resultado obtenido**
P07: archivo inexistente	Ruta a un archivo que no existe	Rechazar apertura; salida 1	Archivo no disponible; salida 1
P08: Tierra ausente	Miller y Mann sin Tierra	No encontrar Tierra; salida 1	Tierra no encontrada; salida 1
P14: constante inválida	$k = 0$	Rechazar datos iniciales; salida 1	Datos iniciales inválidos; salida 1
P10: factor nulo	Miller con $t = 0$	Rechazar Miller; salida 1	Miller rechazado; salida 1
P15: factor menor que uno	Miller con $t = 0.5$	Rechazar Miller; salida 1	Miller rechazado; salida 1

Table 11 Plan de pruebas: planetas y conexiones peligrosas inválidas.

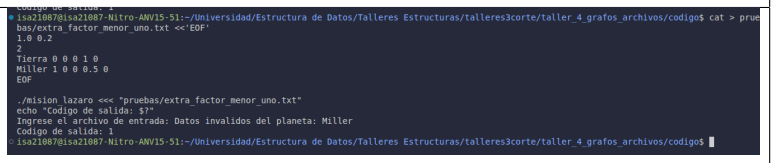
Descripción del caso	**Valores de entrada**	**Resultado esperado**	**Resultado obtenido**
P09: planeta repetido	Dos planetas llamados Tierra	Rechazar duplicado; salida 1	Duplicado rechazado; salida 1
P11: autoconexión peligrosa	Tierra declara peligrosa Tierra–Tierra	Rechazar autoconexión; salida 1	Conexión inválida; salida 1
P05: conexión no recíproca	Tierra declara Miller, pero Miller no declara Tierra	Rechazar conexión; salida 1	No recíproca; salida 1
P06: conexión repetida	Una misma conexión peligrosa aparece dos veces	Rechazar repetición; salida 1	Conexión repetida; salida 1

Las validaciones de los nueve casos coincidieron con el comportamiento esperado. La prueba 15 merece especial atención: un factor temporal de 0.5 debe rechazarse porque el enunciado exige $t \geq 1$, aunque el valor sea positivo. Las evidencias de las Tablas 12 y 13 respaldan estos rechazos. Se conservan también las capturas de los demás errores, pero no se repiten todas en el informe porque muestran el mismo tipo de comprobación: el mensaje correspondiente y la terminación controlada.

Table 12 Evidencia de validación: conexiones peligrosas inválidas.

Descripción del caso	Evidencia de ejecución
P05: conexión no recíproca	
P06: conexión duplicada	

Table 13 Evidencia de validación: datos de planetas inválidos.

Descripción del caso	Evidencia de ejecución
P09: planeta duplicado	
P15: factor temporal $t = 0.5$	

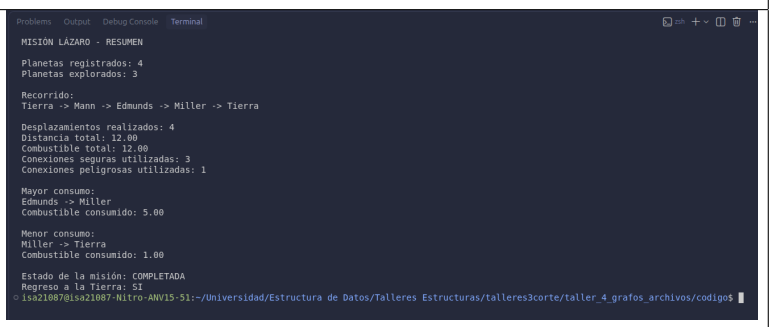
4.6.7 Plan de pruebas: generación de los archivos de salida

Finalmente verificamos que los archivos `detalle_recorrido.txt` y `resumen.txt` correspondieran al recorrido ejecutado. En el primero se muestran los desplazamientos individuales con sus factores, distancias, combustible y acumulados. El segundo resume el total y los tipos de conexiones utilizados.

Table 14 Plan de pruebas: archivos de salida.

Descripción del caso	Valores de entrada	Resultado esperado	Resultado obtenido
P01: resumen básico	Cuatro planetas sin peligros	Cuatro movimientos, cuatro seguras, 18.00	Cuatro movimientos, cuatro seguras, 18.00
P02: detalle temporal	Miller con $t = 8$, $\lambda = 0.5$	Acumulados 3.00, 5.75, 16.75, 22.75	Acumulados 3.00, 5.75, 16.75, 22.75
P03: regreso peligroso	Tierra–Miller peligrosa	Tres seguras, una peligrosa, 12.00	Tres seguras, una peligrosa, 12.00
P04: conexiones peligrosas	Sin salida segura inicial desde Tierra	Una segura, dos peligrosas, 6.00	Una segura, dos peligrosas, 6.00

Table 15 Evidencia del resumen de misión con regreso peligroso.

Descripción del caso	Evidencia de ejecución
P03: regreso peligroso registrado	

En los cuatro casos, los totales del archivo de resumen coincidieron con las sumas de los desplazamientos. El detalle de P02 presentado en la Tabla 4 permite verificar el acumulado después de cada movimiento. El resumen de P03 muestra el regreso peligroso como parte de la misión.

4.6.8 Resultado de las pruebas

Al comparar los valores esperados con los obtenidos, las ejecuciones presentadas en los planes anteriores resultaron satisfactorias. Se comprobó el cálculo del combustible, el cambio del recorrido por el factor temporal, la prioridad de las conexiones seguras y el uso de conexiones peligrosas cuando no hay alternativas. También se verificaron el regreso a Tierra, los empates, el caso de un único planeta y el rechazo de las entradas inválidas ensayadas.

Se conservaron todas las capturas de las ejecuciones, aunque en el informe se incluyeron las más representativas para evitar repetir imágenes con información equivalente. Las evidencias seleccionadas se presentan en tablas con la descripción del caso a la izquierda y la captura correspondiente a la derecha, según la presentación del Taller 0. Las tablas más extensas se dividen para que el texto de la terminal se mantenga legible.